Cost-Aware Multi-Model Inference Router (CAMIR)

Project Proposal (Version 3)

Abhishek Darji Aniket Anil Naik Tamizh Selvan Manivannan

San José State University — MS Software Engineering

Advisor: Prof. Vijay Eranti

October 28, 2026

Abstract

Routing and cascading send easy prompts to a small language model and hard ones to a large model. CAMIR builds and evaluates such a router on a single Apple M4 laptop, with local small and medium tiers and a hosted large tier. The project makes four contributions: (1) a measured account of the *cost of escalation itself*—the cost, delay and energy wasted when a small-model attempt is rejected—and policies that reduce it; (2) a new family of escalation *confidence signals*, including signals read from a mixture-of-experts router; (3) *calibrated routing with guarantees*, a risk-controlled threshold that bounds the quality lost relative to always using the large model, for policies that skip or abort the small attempt; and (4) *per-step routing in agentic workflows*, where switching tiers mid-trajectory carries its own cost. For each contribution we state what prior work already establishes and what remains open. Two extensions sharpen the first contribution: *verification-based escalation*, where the large model scores the small model's answer in one prefill-only pass instead of regenerating it, and a *regime map* that tells a practitioner when cascading, pre-routing or always using the large model wins. Supporting studies test the policies under concurrent load and on open-ended chat traffic, make the guarantee adaptive to distribution shift, and release the trace dataset. The evaluation uses a broader and noisier data plan (controlled perturbations of existing prompts, harder verifiable sets, an instruction-following set and an open-ended chat slice). The system is designed to run on several hardware profiles (Apple Silicon, CPU only, NVIDIA GPU and a cloud instance) so that the regime map can show how the crossover moves with the hardware. Finally, the result is packaged as a pip-installable library with a calibrate, shadow and serve workflow, so that others can use it without cloning the repository.

1 Introduction

Large hosted models are accurate but costly and slow; small local models are cheap and fast but fail more often. Pre-generation routers [2, 4] and confidence-gated cascades [1, 5, 6, 8] try to capture the savings, and cascade routing [3] combines the two. Vendors now ship routers as products [29, 30, 31, 32], and API gateways such as LiteLLM include a pre-generation tier router [23]. The project runs on a MacBook M4 with a pay-per-token API for the large tier: no HPC or cloud credits are needed.

2 Problem Statement

Given a stream of prompts (and, in the agentic setting, multi-step tasks) and a pool of tiers (small, medium, large), choose for each prompt or step which tier answers, so that quality stays within ϵ

of always using the large tier while cost, latency (mean and tail) and energy are minimized, and so that the quality statement is backed by a statistical guarantee rather than a tuned threshold. Published evaluations mostly report token cost or the share of calls sent to the large model [1, 2, 7]; surveys note the lack of shared metrics, pools and cost bases [17] and of non-financial costs [18].

3 Contributions and Novelty

Honest positioning. Several of these topics are already active. Bouchard [7] characterizes threshold cascades decision-theoretically and finds that a pre-generation router often beats the best cascade, attributed mainly to avoiding the cheap model's cost. Conformal methods for routing and cascading already exist [9, 10, 11]. Budget-aware per-step routing for agents exists [13] (and [14], which we have not yet read). We therefore state, for each contribution, a narrower gap that we have not found covered in the sources we read; our literature search is not exhaustive and each claim is to be re-checked with the advisor before submission.

3.1 C1: The cost of escalation itself

Established. A cascade pays for every model it queries [1, 10], and the cheap model's cost explains much of the router's advantage in token cost [7].

Our contribution. With c_S, c_L the tier costs, g the gate cost, e the escalation rate and $\rho = (c_S + g)/c_L$, a cascade saves

$$\text{Savings} = (1 - e)\Delta - eW,$$
$$\Delta = \mathbb{E}[c_L - c_S - g \mid \text{acc}], \quad W = \mathbb{E}[c_S + g \mid \text{esc}],$$

breaking even at $e^* = \Delta/(\Delta + W)$ $(= 1 - \rho$ for homogeneous costs). Per prompt, attempt the small model first only if the predicted acceptance probability $\hat{a}(x) > \rho(x)$. For latency, escalated requests pay $t_S + t_g + t_L$, so under idealized assumptions a cascade's p95 beats always-large only if $e < 5\%$. These relations are elementary; the contribution is (i) measuring the tax in dollars, seconds (mean and tail) and joules on real hardware with a local-plus-hosted pool, and (ii) two policies built from it: **P3**, a skip-rule route-then-cascade, and **P4**, an early-abort cascade that abandons the small attempt when the running mean log-probability falls below a threshold after n_0 tokens.

Extension X1: verification-based escalation (policy P5). In a standard cascade, escalation makes the large model regenerate the answer from scratch, so every rejected attempt is pure waste. P5 changes what escalation does. A verifier tier V reads the prompt and the small model's answer in a single forward pass and scores the answer by the mean log-probability it assigns to it, $v(x, y_S)$. If $v \geq \tau_v$ the small answer is kept; otherwise the large model regenerates. The verifier is the hosted large model if its API returns log-probabilities for supplied text, and otherwise the local medium tier. Verification is prefill-only (no token-by-token decode), so its cost c_V is typically below c_L for long answers. Let v_{pass} be the fraction of escalated requests that pass verification. An escalated request then costs $c_S + g + c_V + (1 - v_{\text{pass}})c_L$ instead of $c_S + g + c_L$, so P5 helps exactly when $v_{\text{pass}}\, c_L > c_V$; it attacks the cases where the small answer was right but the gate was unsure. The idea is related to speculative decoding [21], where a large model verifies a small model's draft in parallel; the differences are that we verify whole answers rather than draft tokens, that acceptance is approximate (its quality loss is controlled by C3, not by an exactness argument), and that the verifier may sit across an API boundary. In the sources we read we have not found answer-level verification charged inside a cascade cost model. Feasibility depends on the log-probability access in the hosted API, which is checked in spike S6; the verifier can also be a local tier, which makes

the medium tier a requirement. A verifier that agrees too easily with wrong answers is a real risk, and E10 measures it.

Extension X2: a regime map. The break-even relations give a decision rule that practitioners need before they build anything. For each point in the plane of cost ratio ρ and escalation rate e, and for a stated latency target, we compute which strategy is cheapest at matched quality (always-large, pre-generation router, cascade, P3, P4, P5), using the measured Δ, W and latency distributions, and we validate the predicted regions by replaying the traces. The deliverable is a figure and a small calculator that takes a team's own e, ρ and latency target. A map that says "do not cascade here" is as useful as one that says "do".

3.2 C2: A new confidence signal

Established. Token log-probability, top-1/top-2 margin (with isotonic calibration [8]), sampled-answer agreement [19, 6], self-verification [5], and probes on hidden states or prompt embeddings are used to decide escalation.

Gap. Mixture-of-experts (MoE) routers produce a per-token distribution over experts. Prior work studies router uncertainty to improve MoE fine-tuning [15], and, in the sources we read, does not use it as a signal for whether an answer is correct or should be escalated.

Our contribution. We test the hypothesis that expert-routing statistics (normalized router entropy, its change across layers, and expert-selection instability) predict small-tier correctness. The small tier would be a small open MoE model (candidates such as OLMoE [16], to be confirmed in a feasibility spike for MLX support and router access). The signal is available at prefill and at decode time at near-zero extra cost, which makes the skip rule and early abort of C1 cheaper to apply. It is compared on AUROC and calibration error against log-prob, margin, self-consistency, and a learned hidden-state probe (our fallback if no suitable MoE model runs locally). A negative result is a reportable outcome.

3.3 C3: Calibrated routing with guarantees

Established. Conformal Cascade [9] defers when a conformal prediction set is not a singleton, with a coverage guarantee (shown on multiple-choice benchmarks; open-ended generation left for future work). RouteNLP [10] gives a marginal conformal guarantee on a violation probability. RACER [11] gives distribution-free risk control for set-valued routing. Conformal risk control [12] is the general tool.

Gap. We have not found a guarantee that covers the full *tax-aware* policy (skip, early abort and gate together), and generation tasks (math word problems, code with unit tests) are less covered than multiple-choice ones.

Our contribution. Let $A(\lambda)$ be the event that the small answer is accepted under threshold λ after any skip or abort rule, and define the loss $\ell(\lambda) = \mathbf{1}[A(\lambda) \wedge S \text{ wrong} \wedge L \text{ right}]$, which is non-increasing in λ and zero when the small answer is not accepted. Since the error of the cascade minus the error of always-large is at most ℓ, choosing $\hat{\lambda}$ by conformal risk control on a calibration split gives $\mathbb{E}[\ell(\hat{\lambda})] \leq \alpha$, i.e. an expected quality loss relative to always-large of at most α. The guarantee is marginal (not per prompt), assumes exchangeability, and weakens under distribution shift; we test coverage empirically with repeated random splits and under a deliberate shift (calibrating on one dataset and testing on another). With P5, acceptance $A(\lambda)$ also includes passing verification; we use a one-parameter family in which the verification threshold is tied to λ, so that acceptance stays monotone in λ and the same argument applies.

Extension X3: adaptive recalibration. A fixed threshold calibrated once is fragile when the prompt mix drifts. We add an online update in the spirit of adaptive conformal inference [22]: a random fraction of requests is audited by also running the large tier, which reveals the loss ℓ, and the working risk level is adjusted after each audited request. The audit rate is a cost, so we report it, and we measure how quickly coverage recovers after a deliberate shift (calibrate on science questions, then stream code and math). We do not claim the finite-sample marginal guarantee for this variant; what we test is whether the long-run average of the audited loss stays near α under shift, and whether an analogous formal statement holds is left to be checked with the advisor.

3.4 C4: Per-step routing in agentic workflows

Established. Budget-aware per-step routing for multi-step agents, trained with boundary-guided reinforcement learning, reduces dollar cost on agent benchmarks [13]; its cost is token dollars and router overhead is excluded from the reported metrics.

Gap. In an agent loop the context grows each step, so moving a step from the local tier to the hosted tier re-sends the whole history (the hosted model must prefill it again), and each tier switch adds latency. We have not found this switching cost modeled or measured on a local-plus-hosted pool.

Our contribution. We extend the C1 model to a trajectory: the cost of escalating at step t includes the wasted small-model step and a prefill term that grows with context length, and a policy chooses the tier at each step. We evaluate on a lightweight multi-step tool-use benchmark that fits a laptop (candidates: function-calling data such as xLAM, or a small multi-step subset of an agent benchmark; feasibility is checked in a spike). We compare per-step policies (always-small, always-large, per-step threshold from C3, a learned step router in the style of [13]) on task success, dollars, seconds and joules.

3.5 Supporting studies and release

Behavior under load. A single-stream laptop study misses queueing delay, which often dominates tail latency in serving. We run a trace-driven discrete-event simulation of concurrent requests, using the measured service-time distributions of each tier, a concurrency or rate cap on the hosted tier, and a single local stream on the M4, and report how the escalation tax shifts p95 and p99 latency at several arrival rates. A small live run with several concurrent clients on a subset checks the simulation.

Open-ended traffic. The four benchmarks have checkable answers, which real traffic often lacks, and published router evaluations show that routing behaves differently on chat prompts [23]. We add a slice of about 200 open-ended chat prompts from a public chat dataset (a candidate is WildChat; its license is checked first). Quality is judged by blinded pairwise comparison against the large model's answer, with a judge from a different model family, both presentation orders, and conflicting verdicts recorded as ties; the judge's self-disagreement is reported. The guarantee on this slice uses the judged loss and is therefore weaker than on the verifiable sets.

Trace release. The trace dataset (outputs, log-probabilities, verification scores, timings, energy, and useful-versus-wasted labels) and the replay harness are released, subject to the licenses of the benchmarks and the hosted provider's terms on redistributing outputs, which are checked before release.

3.6 Priority and fallback

Core: C1 with X1 and X2, C3, the perturbed and harder datasets of Section 8 (needed to test noise robustness and to tighten the guarantee), the portability layer of Section 9 (a design requirement, cheap if done early), and a minimal pip package (library, CLI and no-key mock mode, Section 10). **Supporting:** the load study, the open-ended chat slice, X3, a second hardware profile, the OpenAI-compatible proxy and web regime map, and the trace release. **Higher risk:** C2 and C4. If no MoE model runs locally, C2 falls back to a hidden-state probe; if no suitable agent benchmark runs on the laptop, C4 is restricted to multi-step chains assembled from the same datasets; if the hosted API cannot score supplied text, X1 uses the local medium tier as verifier, so the medium tier is kept. If time runs short, scope is cut in this order: C4, C2, the second hardware profile, X3, the open-ended slice, the load study, the proxy and container extras. C1, C3, X1, X2, the perturbed and harder datasets, and the minimal package are not cut. Whether the extensions beat the baselines is open; null results will be reported.

Contribution	Closest prior work	Open gap addressed
C1 Escalation cost (+X1, X2)	[1, 7, 10, 21, 23]	Latency, tail, energy on local+hosted pool; tax-aware policies; answer-level verification instead of regeneration; regime map
C2 Confidence signal	[8, 19, 5, 15]	MoE router statistics as escalation signal
C3 Guarantees (+X3)	[9, 10, 11, 12, 22]	Guarantee for skip/abort/verify policies; generation tasks; audited recalibration under shift
C4 Agentic per-step	[13, 14]	Switching/re-prefill cost across steps; measured latency and energy

4 Architecture

Client layer (dashboard, offline harness) → FastAPI router service (ingestion and cache, feature extractor, policy engine, confidence-signal module, conformal calibrator, cost and latency accountant, and a step controller for agent loops) → tiers (small and medium MLX-LM 4-bit local, large hosted API) → trace store (SQLite/Parquet). Evaluation is trace-driven: outputs, signals, log-probs, timings and energy are cached per prompt (or per step) and tier, policies are replayed offline, and a live subset validates replay fidelity. Caching every tier's output for every prompt is also what makes the counterfactual losses in C3 computable. For P5 the traces also store the verifier's score of each small answer, and for the load study they store each tier's service-time distribution. Local tiers sit behind one backend interface (MLX-LM, llama.cpp, optionally vLLM [28]), and every trace row records its hardware profile.

5 Two Paths

Path A (local, medium tier) Small and medium local tiers; routing between them; baselines; proof of concept; signal study (C2) on local models; started in CMPE 295A.

Path B (large tier) Adds the hosted large tier and real cost accounting; the guarantee experiments (C3), tax-aware policies and agentic extension (C4); completed in CMPE 295B.

Both paths share the same code, trace format and harness.

6 Features

- Pluggable policies and baselines (always-S/M/L, random, feature router, a complexity tier router modelled on LiteLLM and re-implemented from its documentation, cascades, cascade routing, oracle, P3, P4, P5, per-step policies).
- A regime-map calculator and a load simulator driven by the same traces.
- A pip-installable library and command-line tool with calibrate, shadow and serve commands, an OpenAI-compatible proxy and a no-key mock mode (Section 10).
- Confidence signals: mean log-prob, minimum token probability, margin, self-consistency, self-verification, MoE router statistics, hidden-state probe.
- Conformal calibrator that outputs a threshold and its stated guarantee for a chosen α.
- Per-request and per-step accounting of cost, TTFT/TPOT/end-to-end time and energy, split into useful and wasted work.
- Trace store, replay harness, statistics (bootstrap intervals, paired tests), frontier plots, and a web dashboard.

7 Requirements

- Hardware: Apple M4 MacBook (variant and RAM recorded in setup) as the primary profile. Hosted large-tier API with a small spending cap. Optional second profile: CPU only, an NVIDIA GPU (for example a teammate's Windows laptop through WSL2), or a short, capped cloud session.
- Software: MLX-LM, llama.cpp/Ollama (fallback and cross-platform backend), optionally vLLM [28], FastAPI, SQLite, Parquet, optional Redis, a sandbox for HumanEval; for packaging, uv, hatchling, GitHub Actions, TestPyPI and PyPI.
- Models: Qwen2.5 small ≈1.5B, medium 7B (4-bit), large 72B hosted [20]; a small open MoE model for C2; Llama 3.x fallback.
- Data: the core benchmarks plus the noisy, harder, instruction-following and chat sets described in Section 8 (target of about 3,000 prompts in total, set by the hosted-call budget), a multi-step tool-use set for C4, and a judge model from a different family for the open-ended slice.

8 Datasets and Data Plan

The four core benchmarks have checkable answers and are cleaner than the prompts real users write. Their size also limits the guarantee: with $n = 272$ calibration prompts the conformal bound carries slack of about $1/(n+1) \approx 0.4$ points, and the standard error of a 5% loss rate is about 1.3 points, so a target α below roughly 3% would not be meaningful. The data plan therefore adds noisier, harder and larger sets, and keeps ground truth wherever possible so that no extra judge noise is introduced.

Source	Role	Ground truth	Notes
ARC-Easy, MMLU, GSM8K, HumanEval (1,364 prompts, built)	Core benchmarks	Option, number, unit tests	Low noise; may overlap pretraining data
Perturbed twins (about 300 base prompts, 2–3 noise types)	Noise robustness (E14)	Inherited from the base prompt	Synthetic noise; paired with the clean prompt
Instruction-following set (IFEval-style [25], about 500 prompts)	Constraint following	Scripted checkers	Tests following constraints, not knowledge
Harder checkable sets (MATH-500 levels 4–5, MMLU-Pro [26], a BigCodeBench [27] subset)	Cases where the small tier often fails	Exact match, unit tests	More hosted-call cost; code needs the sandbox
Open-ended chat slice (200–300 prompts; WildChat [24] is a candidate)	Realism	None; blinded pairwise judge	Natural noise; license, personal data and judge noise
Multi-step tool-use set (C4)	Agentic routing	Task success	Feasibility checked in spike S4

Controlled noise. Perturbations are applied to the prose of a prompt only, so the answer cannot change: for multiple-choice items the stem is perturbed and the options are left intact; for GSM8K the numbers are untouched; for HumanEval only the docstring prose is changed and the signature and tests are kept. Noise types are (i) character typos, swaps and dropped letters at a fixed rate, (ii) lowercase text with stripped punctuation, (iii) irrelevant filler or context added before the question, and optionally (iv) a casual rewrite by a model, checked by a spot audit. The generator is deterministic and versioned (fixed seeds). Because each noisy prompt has a clean twin, the study measures directly how accuracy, gate quality and the escalation rate change with noise, for each tier and each noise type. Whether small models lose more than large ones under noise is a hypothesis to test, not an assumption.

Splits. Splits are stratified by source and by dataset. A clean prompt and its noisy twins always fall in the same split, because otherwise the calibration data would leak into the test set. One whole source (for example the instruction-following set or the chat slice) is held out of calibration to serve as the dataset-shift test in E8. The calibration split is made as large as the budget allows, and the conformal slack is reported with every guarantee. Results are reported per source as well as pooled, because a pooled average can hide where the gate fails.

Threats and limits. The core benchmarks may appear in the pretraining data of the Qwen models, so we rely on relative comparisons between policies and treat the harder and newer sets as partial protection. Dataset licenses and personal data in chat logs are checked before use; raw chat logs are not redistributed. The corpus size is set from the real hosted price (after spike S2): each added prompt costs a generation call and a verification pass, so every set is first run on a 20-prompt pilot and the bill is checked before the full run.

9 Portability Across Environments

The cost ratio ρ, the escalation rate and the tail latency all depend on the hardware: a CPU-only machine makes the local tier slow and raises ρ; an NVIDIA GPU makes it fast; a cloud instance turns free local compute into dollars per GPU-hour. A regime map that shows how the crossover moves across hardware is therefore more useful than one measured on a single laptop. We treat portability as a design requirement and one extra environment as an experiment.

Design (core). Every trace row and the configuration hash carry a `hardware_profile` field. Local tiers sit behind one backend interface with MLX-LM (Apple Silicon), llama.cpp (CPU, CUDA and Metal) and optionally vLLM [28] (NVIDIA and cloud). Energy measurement is a per-profile plug-in: `powermetrics` on Apple Silicon, NVML on NVIDIA GPUs, RAPL where a CPU exposes it, and none on a cloud instance, where results are limited to dollars and time. Local-tier traces are re-collected for each profile, because quantization formats and kernels differ and outputs are not identical; the hosted large tier is shared.

Experiment (supporting, E15). We repeat the local-tier trace collection on one additional profile and test whether the regime-map boundaries move. The cheapest candidate is CPU-only inference with llama.cpp on the same laptop; an NVIDIA run follows if a team member has the machine (Windows through WSL2); a cloud run is a short capped session, and only if time remains. We do not claim performance parity across platforms.

10 Packaging and Distribution

The target audience is developers and small teams that already call an LLM API; a non-technical end user needs a desktop application, which we list as future work. The goal for this audience is that they change one line and run one command, without cloning the repository.

- **Library and CLI.** `pip install camir` installs the core; extras add backends (`camir[mlx]` for Apple Silicon, guarded by a platform marker, and `camir[llama]`). A `camir` command exposes the workflow below.
- **OpenAI-compatible proxy.** `camir serve` runs a local server; an application only changes its `base_url`.
- **Calibration workflow.** The guarantee holds only for a workload similar to the calibration data, so configuration must include calibration. `camir calibrate` takes a few hundred of the user's own prompts, runs the tiers, fits the threshold for a chosen α and writes a configuration file. `camir shadow` runs next to real traffic without routing anything and reports the projected savings and quality loss. `camir serve` then turns routing on, falls back to the large tier on any error, and can sample audits (X3).
- **Mock mode.** A mode that needs no API keys or models, so a quickstart works in minutes and the test suite runs anywhere.
- **Regime-map web page.** A static page where a team enters its cost ratio, escalation rate and latency target and reads the recommendation; no installation.
- **Optional.** A container image for the CPU and NVIDIA backends (as far as we know, Linux containers on a Mac cannot use the Metal GPU, so Apple users run natively; to be checked) and a plug-in for an existing gateway such as LiteLLM's custom routing hook (not yet tested).

Release practice. The package name is checked on PyPI first. The wheel is built with hatchling or uv and installed in a clean environment before upload; releases go to TestPyPI first and then to PyPI through trusted publishing from a tagged GitHub release. PyPI versions cannot be re-uploaded, so each version is released deliberately. The wheel contains code and an example configuration only; no traces, prompts, API keys or model files. The release is archived with a DOI for the thesis, and the license is chosen after checking the model and dataset terms. The README states the limits plainly: the guarantee is marginal, assumes similar prompts, and has been tested only on the benchmarks above.

11 Evaluation Plan

E1 per-tier characterization; E2 signal quality (AUROC, calibration error) for all signals including C2; E3 tax measurement; E4 policy frontiers; E5 sensitivity to ρ; E6 tail latency; E7 replay-versus-live fidelity; E8 guarantee coverage (violation rate against α over repeated splits and under dataset shift); E9 agentic per-step policies (task success, cost, time, energy, switching overhead); E10 verification-based escalation (P5 against P2–P4: pass rate v_{pass}, c_V/c_L, wrong answers accepted by the verifier, net savings); E11 regime map (predicted strategy regions against replayed ones); E12 behavior under load (p95/p99 against arrival rate, simulation against a small live run); E13 open-ended chat slice (gate quality and policy frontiers without ground truth, judge agreement); E14 noise robustness (clean versus perturbed twins: change in accuracy, gate AUROC and calibration, and escalation rate, per tier and noise type); E15 cross-environment regime map (how the crossover moves across hardware profiles, using re-collected local-tier traces); E16 packaging check (install in a clean environment on macOS and Linux, run the mock quickstart, and run shadow mode on a sample workload); E8 is extended with adaptive recalibration under shift and its audit cost. Quality must be within ϵ (2–5 points) of always-large. Dollars, joules and seconds are reported separately. Threats: thermal throttling, benchmark contamination, API variance, replay fidelity, small calibration sets. No results exist yet.

12 Timeline (two semesters)

Weeks	Milestone
295A 1–3	Abstract, formation presentation, topic and advisor sign-off
295A 3–9	Literature and baselines; feasibility spikes (S1–S3 done; verifier access S6; MoE and agent feasibility S4); trace schema, tier interface, portability layer and package skeleton (layout, CLI stub, mock mode); architecture and design
295A 9–15	Functional specifications; proof of concept: expanded dataset build (perturbed twins, instruction-following and harder sets, chat slice), full trace collection including verification scores, scorers, replay engine, baselines, E1 and the replay-versus-live check; report chapters 1–2
295B 1–7	Live router; accountant; tax measurement (E3); P3 and P4; conformal calibrator; noise robustness (E14)
295B 5–10	P5 verification-based escalation (E10); coverage and adaptive recalibration (E8); regime map (E11); chat-slice judging (E13)
295B 8–12	Load study (E12); second hardware profile (E15); calibrate, shadow and serve workflow; C2 and C4 if time allows

Weeks	Milestone
295B 11–14	Packaging check (E16) and TestPyPI then PyPI release; testing and performance analysis; trace release; report; advisor reviews
295B 14–15	Presentation, poster, Project Exposition, final report

13 Deliverables

Router service; pip-installable library and CLI (calibrate, shadow, serve) with an OpenAI-compatible proxy and a no-key mock mode; trace dataset with useful-versus-wasted labels; build scripts for the expanded corpus (perturbation generator, instruction-following and harder sets, chat slice); evaluation harness; tax-aware policies including verification-based escalation; regime-map calculator and web page; load simulator; portability layer with at least one additional hardware profile; confidence-signal library including the MoE-router signal; conformal calibrator with adaptive recalibration; agentic step router; dashboard; analysis report; paper draft; open-source release archived with a DOI.

14 Risks

- Log-probs or MoE router outputs unavailable in MLX-LM: feasibility spikes; fall back to llama.cpp/Ollama, or to a hidden-state probe for C2.
- Hosted API price or availability changes: cache all large outputs; two providers.
- Thermal throttling and API variance distort latency: repetition, logging, cooldown.
- Overlap with fast-moving related work (conformal routing, agentic routing): re-check the literature monthly and narrow the claims if a gap closes.
- The hosted API cannot score supplied text, or the verifier accepts wrong answers too often: use the local medium tier as verifier and report the verifier's error rate (spike S6, E10).
- Judge noise on the open-ended slice: both presentation orders, ties for conflicts, a judge from another model family, and the judge's self-disagreement reported.
- Synthetic noise is unrealistic or alters the answer: perturb prose only, spot-audit samples, keep clean twins, and use the chat slice as the realism check.
- The calibration split is too small for a tight guarantee: grow it, report the conformal slack, and avoid very small α.
- Dataset licenses and personal data in chat logs: check licenses before use, scrub, and do not redistribute raw logs.
- Backends differ across hardware (quantization, kernels, energy counters): re-collect local traces per profile, label results per profile, and do not claim parity.
- Packaging: PyPI releases are permanent, MLX is Apple-only, and containers on macOS may lack GPU access: test on TestPyPI and in a clean environment, and use extras with platform markers.

- Scope: four contributions, three extensions, a broader data plan, portability and packaging in two semesters is ambitious; the cut order is C4, C2, the second hardware profile, X3, the open-ended slice, the load study, the proxy and container extras. C1, C3, X1, X2, the perturbed and harder datasets and the minimal package are protected.
- The tax proves small or routers equal cascades: report it as a finding.

15 Expected Outcomes

A measured account of when the cost of escalation makes cascading worse than routing in cost, latency and tail latency on commodity hardware, summarized as a regime map; a test of whether verifying the small answer instead of regenerating it lowers that cost; an evaluation of whether MoE router statistics are a useful escalation signal; thresholds with a stated, empirically tested quality guarantee for tax-aware policies; a first measurement of tier-switching cost in agent loops; evidence on how noise and hardware change the answer; and an open, reproducible system that others can install with pip, calibrate on their own prompts and run in shadow mode before routing real traffic.

References

[1] L. Chen, M. Zaharia, J. Zou. FrugalGPT. arXiv:2305.05176, 2023.

[2] I. Ong et al. RouteLLM. arXiv:2406.18665, 2024.

[3] J. Dekoninck, M. Baader, M. Vechev. A unified approach to routing and cascading for LLMs. ICML 2025. arXiv:2410.10347.

[4] D. Ding et al. Hybrid LLM. ICLR 2024. arXiv:2404.14618.

[5] P. Aggarwal et al. AutoMix. NeurIPS 2024. arXiv:2310.12963.

[6] M. Yue et al. LLM cascades with mixture of thought representations. ICLR 2024. arXiv:2310.03094.

[7] D. Bouchard. Is escalation worth it? A decision-theoretic characterization of LLM cascades. arXiv:2605.06350, 2026.

[8] V. Kotte. UCCI: Calibrated uncertainty for cost-optimal LLM cascade routing. arXiv:2605.18796, 2026.

[9] Y. Dou, S. Lian, S. Li. Conformal Cascade: Distribution-free accuracy guarantees for multi-tier LLM inference. arXiv:2607.25018, 2026.

[10] D. Guo, S. M. Yiu, J. Wu. RouteNLP: Closed-loop LLM routing with conformal cascading and distillation co-optimization. arXiv:2604.23577.

[11] S. Hao, H. Zeng, H. Wei, B. Jing. RACER: Risk-aware calibrated efficient routing for large language models. arXiv:2603.06616, 2026.

[12] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, T. Schuster. Conformal risk control. arXiv:2208.02814 (ICLR 2024).

[13] C. Zhang et al. Budget-aware agentic routing via boundary-guided training. arXiv:2602.21227, 2026.

[14] AgentRouter: Heterogeneous model routing for cost-optimal multi-step agentic workflows. arXiv:2609.22951 (not yet read; to be reviewed).

[15] H. Wu et al. GW-MoE: Resolving uncertainty in MoE router with global workspace theory. arXiv:2406.12375.

[16] N. Muennighoff et al. OLMoE: Open mixture-of-experts language models. arXiv:2409.02060, 2024.

[17] Y. Moslem, J. D. Kelleher. Dynamic model routing and cascading for efficient LLM inference: a survey. TMLR. arXiv:2603.04445.

[18] C. Varangot-Reille et al. Doing more with less: a survey on routing strategies for resource optimisation in LLM-based systems. arXiv preprint, 2025.

[19] X. Wang et al. Self-consistency improves chain of thought reasoning. ICLR 2023. arXiv:2203.11171.

[20] Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.

[21] Y. Leviathan, M. Kalman, Y. Matias. Fast inference from transformers via speculative decoding. ICML 2023. arXiv:2211.17192.

[22] I. Gibbs, E. Candès. Adaptive conformal inference under distribution shift. NeurIPS 2021. arXiv:2106.00170.

[23] BerriAI. LiteLLM documentation: Router and Auto Routing, and the auto-routing benchmark page. docs.litellm.ai (vendor-reported results; the auto router was documented as a beta feature when read).

[24] W. Zhao et al. WildChat: 1M ChatGPT interaction logs in the wild. ICLR 2024. arXiv:2405.01470.

[25] J. Zhou et al. Instruction-following evaluation for large language models. arXiv:2311.07911, 2023.

[26] Y. Wang et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. NeurIPS 2024. arXiv:2406.01574.

[27] T. Y. Zhuo et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. ICLR 2025. arXiv:2406.15877.

[28] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. SOSP 2023. arXiv:2309.06180.

[29] Amazon Web Services. Use Amazon Bedrock Intelligent Prompt Routing for cost and latency benefits. AWS Machine Learning Blog, April 22, 2025.

[30] Microsoft. Model router for Microsoft Foundry: How it works. Microsoft Learn documentation.

[31] NVIDIA. Deploying the NVIDIA AI Blueprint for cost-efficient LLM routing. NVIDIA Technical Blog, March 26, 2025.

[32] OpenRouter. Auto Router. OpenRouter documentation.